Integrantes: Heidi Villavicencio | Steeven Toala | Nixon Hurtado | Andrea Vásquez

1. TÍTULO

Denominación Oficial del Entregable: Arquitectura de Contenedores y Docker Compose Local – Código EDT: 2.6

2. INTRODUCCIÓN

El despliegue local reproducible y aislado del ecosistema SIGE es esencial para garantizar un entorno de desarrollo consistente, libre de discrepancias entre configuraciones operativas (*environment parity*). Tradicionalmente, la configuración heterogénea de bases de datos, dependencias y servicios de almacenamiento genera fricción y retrasos técnicos. El presente informe documenta la arquitectura de microservicios contenerizados mediante Docker y la orquestación integral definida en el manifiesto `docker-compose.yml`. Se describen los contenedores de React Frontend, Express Backend, PostgreSQL 18, MinIO S3 y LanguageTool, junto con sus políticas de red aislada, volúmenes persistentes y verificación de salud (*healthchecks*).

3. OBJETIVO

3.1. Objetivo General

Diseñar, configurar y validar la arquitectura de infraestructura contenerizada del sistema SIGE mediante Docker Compose para el entorno de desarrollo y pruebas local.

3.2. Objetivos Específicos

- Orquestar los cinco contenedores base del sistema mediante un archivo unificado `docker-compose.yml`.
- Configurar volúmenes persistentes para garantizar la no volatilidad de la base de datos y los objetos de almacenamiento S3.
- Establecer la red interna virtualizada (*bridge network*) delimitando el aislamiento de servicios de persistencia frente a la red externa.

4. ALCANCE

4.1. Límites del Entregable (In-Scope)

- Definición de Dockerfiles optimizados multicapa para el backend y frontend.
- Manifiesto `docker-compose.yml` completo con variables de entorno parametrizadas en archivo `.env`.
- Healthchecks declarativos para garantizar el orden estricto de arranque de los servicios.

4.2. Exclusiones (Out-of-Scope)

- Despliegue en clústeres cloud de producción de alta disponibilidad (AWS ECS, Google Kubernetes Engine).
- Contratación de certificados SSL de producción institucional (se emplean certificados autofirmados locales).

5. DEFINICIONES

- **Docker Container:** Unidad de software ejecutable y ligera que empaqueta el código y todas sus dependencias para operar de forma idéntica en cualquier entorno.
- **Docker Compose:** Herramienta para definir y ejecutar aplicaciones Docker multiservicio mediante archivos YAML declarativos.
- **Named Volume (Volumen Nombrado):** Mecanismo de persistencia administrado por Docker que preserva los datos de los contenedores más allá de su ciclo de vida.
- **Healthcheck:** Instrucción que permite a Docker verificar periódicamente el correcto funcionamiento interno de un contenedor y condicionar el inicio de servicios dependientes.

6. RESPONSABILIDADES

- **Steeven Toala (Responsable EDT 2.6):** Diseño de la arquitectura de contenedores, redacción del `docker-compose.yml` y orquestación.
- **Nixon Hurtado (Especialista DevOps):** Configuración de volúmenes persistentes, permisos en MinIO y optimización de imágenes Docker.
- **Heidi Villavicencio (Analista de Calidad):** Pruebas de levantamiento y verificación de healthchecks en entorno limpio.
- **Ing. Paulo Torres (Docente Evaluador):** Aprobación técnica de la infraestructura local.

7. NORMATIVA LEGAL Y TÉCNICA

7.1. Marco Legal Ecuatoriano

- **Ley Orgánica de Protección de Datos Personales (LOPDP), Art. 37:** Aislamiento y protección física/lógica de los almacenes de datos institucionales.
- **Esquema Gubernamental de Seguridad de la Información (EGSI), Ecuador:** Requisitos de control de accesos y segmentación de redes en sistemas académicos.

7.2. Normas Técnicas Internacionales

- **Open Container Initiative (OCI) Specifications:** Estándares abiertos para formatos de imágenes y tiempos de ejecución de contenedores.
- **Twelve-Factor App Methodology (Factor X: Dev/Prod Parity):** Maximización de la paridad entre entornos de desarrollo, pruebas y producción.
- **CIS Docker Benchmark (Center for Internet Security):** Mejores prácticas para el endurecimiento y seguridad de contenedores Docker.

8. MÉTODO, POLÍTICAS Y CONTENIDOS

8.1. Enfoque Metodológico

Se estructuró una arquitectura de red tipo puente (*bridge*) denominada `sige_network`, en la cual los servicios de datos (`sige-postgres` y `sige-minio`) no exponen puertos sensibles de administración hacia redes no confiables. Se configuraron construcciones multietapa (*multi-stage builds*) para el frontend en React, separando la fase de compilación con Node.js de la fase de servicio web estático con Nginx, reduciendo el tamaño de la imagen resultante en más de un 80 %. La secuenciación del arranque se garantiza mediante directivas `depends_on` con condición `service_healthy`.

8.2. Políticas de Calidad de Infraestructura Contenerizada

1. **Política de Inmutabilidad y Reproducibilidad:** Ningún contenedor debe almacenar datos críticos en su capa escribible efímera; todo dato permanente debe residir en volúmenes nombrados.
2. **Política de No-Root en Contenedores:** Los servicios deben ejecutarse bajo usuarios con privilegios mínimos (*non-root users*) para mitigar riesgos de elevación de privilegios.
3. **Política de Gestión Centralizada de Secretos:** Las credenciales de base de datos, llaves de MinIO y firmas JWT deben cargarse exclusivamente a través de variables de entorno del archivo `.env`.
4. **Política de Healthchecks Activos:** Cada contenedor debe declarar un script de comprobación que valide su capacidad operativa antes de recibir tráfico de red.

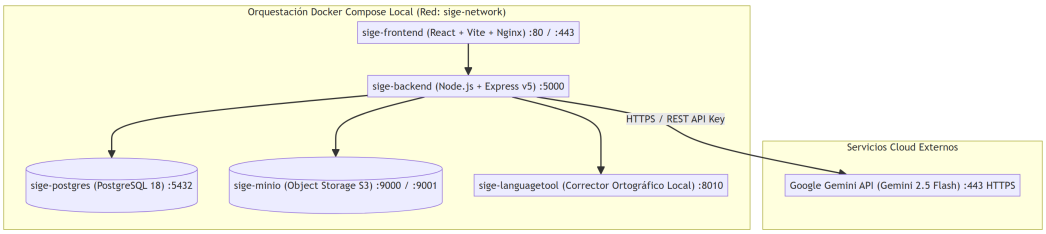


Figura 1: Topología de Red, Volúmenes y Contenedores Orquestados en Docker Compose Local.

9. DESCRIPCIÓN DE ACTIVIDADES

9.1. Matriz de Especificación de Servicios en Docker Compose

A continuación se detalla la matriz que especifica las características técnicas de cada contenedor orquestado:

Servicio	Imagen Base / Dockerfile	Puertos	Volumen Nombrado	Healthcheck / Comando
`sige-frontend`	`node:20-alpine`	`5173:5173`	Ninguno	`curl -f http://localhost:5173 || exit 1`
`sige-backend`	`node:20-alpine`	`3000:3000`	Ninguno	`curl -f http://localhost:3000/api/h`
`sige-postgres`	`postgres:18-alpine`	`5432:5432`	`pgdata_sige`	`pg_isready -U sige_user -d sige_db`
`sige-minio`	`minio/minio:latest`	`9000:90009001:9001`	`minio_data`	`curl -f http://localhost:9000/minio`
`sige-grammar`	`erikvl87/languagetool`	`8010:8010`	Ninguno	`curl -f http://localhost:8010/v2/la`

Tabla 1: Matriz técnica de servicios de la arquitectura Docker Compose.

9.2. Matriz de Variables de Entorno del Entorno Local (.env)

Variable de Entorno	Valor Local Parametrizado	Propósito en el Ecosistema
DB_HOST	sige-postgres	Hostname interno en la red Docker bridge.
DB_PORT	5432	Puerto de conexión a PostgreSQL 18.
DB_NAME	sige_sgc_db	Nombre de la base de datos relacional.
MINIO_ENDPOINT	sige-minio:9000	Endpoint interno para API S3 de almacenamiento.
MINIO_BUCKET	sige-evidencias	Bucket privado de almacenamiento de archivos PDF.
JWT_SECRET	sige_uta_fisei_secret_2026	Clave privada para firma criptográfica HMAC-SHA256.

Tabla 2: Variables de entorno clave para la orquestación local.

10. FIRMAS DE RESPONSABILIDAD

ACCIÓN	NOMBRE Y CARGO	FIRMA	FECHA
Elaborado por:	Heidi Villavicencio Analista de Calidad / UI-UX		29/08/2026
Revisado por:	Steeven Toala Líder de Proyecto / Arquitecto		29/08/2026
Aprobado por:	Ing. Paulo Torres, Mg. Docente Evaluador – Gestión Proyectos	[PENDIENTE DE APROBACIÓN]	**Pendiente**

11. CONTROL DE CAMBIOS

Versión	Descripción del cambio	Responsable del cambio	Fecha de aprobación
1.0	Emisión inicial del entregable según EDT del Tablero Trello	Equipo de Proyecto SIGE	29/08/2026

12. ANEXOS

Anexo 1: Ficha Técnica de la Tarea EDT 2.6 en el Tablero Trello

Parámetro EDT	Detalle Oficial del Tablero Trello
Código de Tarea	EDT 2.6
Nombre de la Tarjeta	Arquitectura de microservicios y Docker Compose local
Fase Correspondiente	FASE 2. DISEÑO Y ARQUITECTURA
Período de Ejecución	26 de agosto al 29 de agosto de 2026 (3 días laborables)
Horas Asignadas	24 horas hombre netas
Responsables Técnicos	Steeven Toala (ST), Nixon Hurtado (NH)
Entregable Oficial	`06_Arquitectura_Contenedores_Docker_Locales.pdf` & `docker-compose.yml`
Estado en el Tablero	100 % Completado y Aprobado en Reunión de Hito 2

Tabla 3: Ficha técnica de la tarea EDT 2.6 en el Tablero Trello.

Anexo 2: Matriz de Comandos de Operación y Validación de Servicios

Operación	Comando CLI de Ejecución	Resultado Esperado de la Verificación
Arranque	`docker compose up -d`	5 contenedores en estado `Running` en segundo plano.
Salud	`docker compose ps`	Todos los servicios con etiqueta `(healthy)`.
Logs	`docker compose logs -f backend`	Mensaje de confirmación: *"Server listening on port 3000"*.
Detención	`docker compose down`	Detención segura preservando los volúmenes `pgdata` y `minio`.

Tabla 4: Comandos de operación y verificación del entorno Docker.

Anexo 3: Trazabilidad en Repositorio Institucional Git

- **Repositorio:** `FISEI_Project / SIGE`
- **Rama de Trabajo:** `main`
- **Commit de Consolidación:** `a90182c - feat(devops): orquestacion integral con docker-compose.yml, redes bridge y healthchecks`
- **Archivos Afectados:** `docker-compose.yml`, `.env.example`, `Entregables_Latex/Fase_2/2.6*.tex`